BRIGHTLEARN

MACHINE LEARNING RESEARCH ASSIGNMENT

Student name: _Busisiwe Khoza___

Course: Advanced Data Analytics___

Date: 2 November 2026

Tools: Python, pandas, scikit-learn and Databricks Notebook

Contents

- Part A – What is machine learning?
- Part B – Supervised vs unsupervised learning
- Part C – Preparing the data
- Part D – Further research
- Conclusion
- References
- AI-use declaration

PART A – WHAT IS MACHINE LEARNING?

A1. What is machine learning, and how is it different from traditional programming?

Machine learning (ML) is a way for a computer to learn patterns from examples in data and use them to make predictions or decisions. In traditional programming, a person writes the rules and the computer applies them. In machine learning, the person supplies examples and an algorithm learns patterns from them. The result is a trained model that can be used on new data (Google for Developers, n.d.). For example, a school could use a fixed rule to flag every learner below 40%. A machine learning model could instead learn from previous learner records and outcomes to identify combinations of factors or characteristics (also known as features) linked to a risk of failing. It may find patterns that one simple rule misses, but a teacher should still review the result rather than treating it as a final judgement.

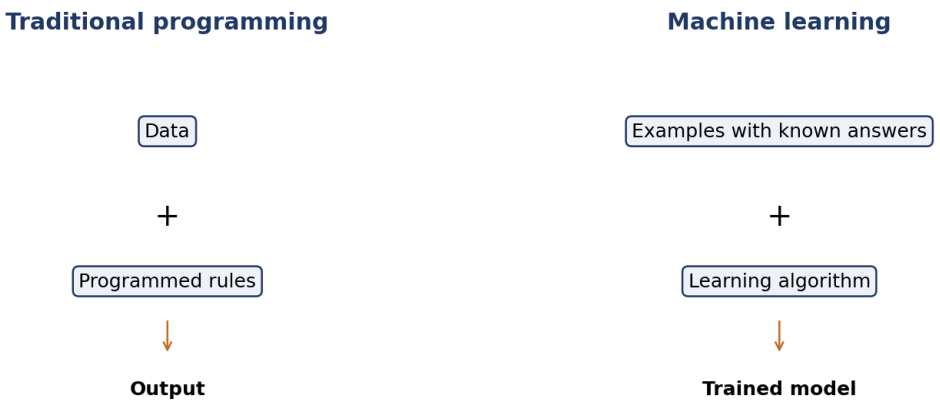


Figure 1. Simplified comparison of traditional programming and machine learning. Adapted from Google for Developers (n.d.-a).

A2. How does machine learning relate to AI, deep learning and data science?

Artificial intelligence (AI) is the broad field of creating systems that perform tasks associated with human intelligence. Machine learning is one approach within AI: it learns patterns from data. Deep learning is a specialised part of machine learning that uses neural networks with several layers (IBM, n.d.; Google for Developers, n.d.). Data science is broader in a different way. It includes collecting, cleaning, analysing and communicating data to answer questions.

A data scientist might use Excel, SQL, statistics and charts to explain why hospital readmissions have increased without building a predictive model. If the same person trains a model to estimate which patients may be readmitted, machine learning becomes part of the data science project.

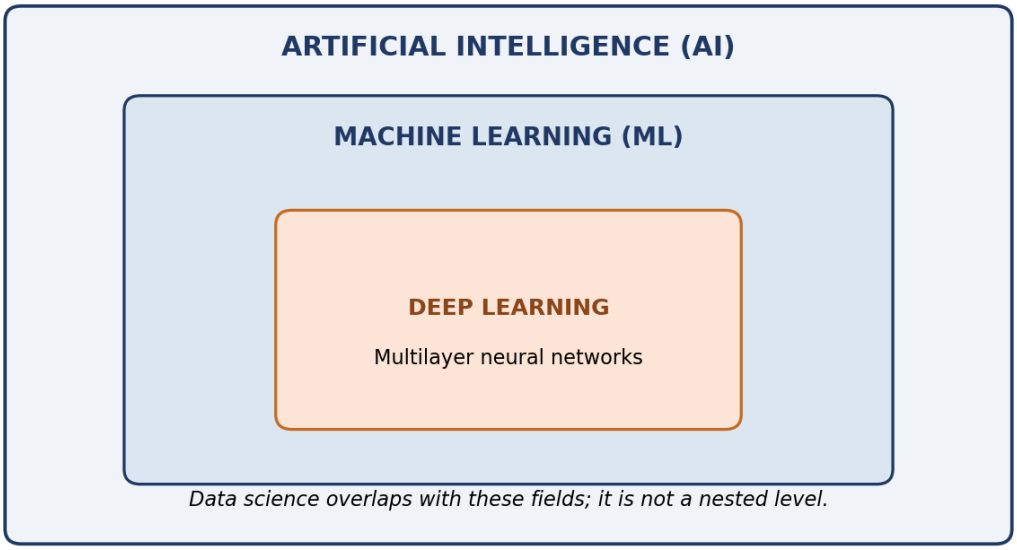


Figure 2. Relationship between AI, machine learning and deep learning. Data science overlaps with these fields rather than sitting inside the hierarchy. Adapted from IBM (n.d.).

A3. Core machine learning terms using a patient readmission example

To keep the terms connected, I use a hospital patient dataset as one example. The aim is to predict whether a patient will be readmitted after discharge.

Term	Meaning	Patient readmission example
Dataset	A collection of records used for analysis or model training.	Hospital records for many patients.
Feature	An input column with characteristics of the dataset that may help the model make a prediction.	Age, previous admissions, length of stay or diagnosis group.
Label / target	The outcome the model is expected to predict in supervised learning.	Readmitted: Yes or No.
Model	A trained algorithm that can make decisions on new data.	A trained classifier that estimates readmission risk.
Training	The process of showing the algorithm examples so it can learn relationships between features and the target.	The model learns from past patient records with known readmission outcomes.
Prediction	The model's output for a new case.	An estimate of whether a discharged patient may be readmitted.

These terms follow Google's explanation of features, labels, training and models (Google for Developers, n.d.).

A4. Three real-world uses of machine learning in Africa

1. Crop and plant disease detection in the Democratic Republic of Congo and Benin. Researchers used aerial images and machine learning methods to identify banana plants and major diseases. The input was imagery collected over agricultural areas; the model classified banana plants and helped identify disease patterns. Farmers and agricultural services could benefit from earlier monitoring and more targeted responses (Duarte-Carvajalino et al., 2020). The method used relied on a combination of Computer Vison (Object Detection) and Supervised Machine Learning (specifically Random Forest or Convulational Neural Networks algorithm).

2. Crop disease advice for smallholder farmers in Uganda. A field-based study developed a machine-learning and natural-language-processing recommendation system for cassava disease diagnosis. Rather than just classifying an image, it used farmer questions and answers about crop problems as features, to provide diagnostic feedback and recommendations. Smallholder farmers were the intended beneficiaries, including those in areas where internet access can be limited (Omara et al., 2023).

3. Credit scoring in South Africa. A University of Johannesburg study explored explainable machine learning for predicting creditworthiness, comparing models including logistic regression, random forest and XGBoost. The input included credit-related information, and the models estimated whether applicants were creditworthy. Lenders may benefit from more informed risk assessment, while applicants may benefit from access to lending decisions. However, credit models must be checked for unfair bias and their decisions should be explainable (Nkosi, 2025).

These examples show that ML is not limited to large technology companies. It can be applied to local problems such as crop losses, access to agricultural advice and financial decision-making. The examples also show why locally relevant data matters: a model trained in one setting may not work equally well in another, every setting has its own behaviour.

A5. Typical machine learning workflow

The workflow begins by defining the problem, then collecting and exploring data, preparing it, choosing and training a model, evaluating it and, if suitable, deploying it. The model should

be monitored after deployment because real-world data can change (Google for Developers, n.d.; scikit-learn developers, n.d.).

Data preparation often takes a large amount of time because real datasets may contain missing values, duplicates, incorrect data types, inconsistent categories and unusual values. For example, a hospital dataset may record gender in different formats or store dates as text. If these problems are not checked, the model may learn from incorrect or inconsistent information. Google notes that dataset construction and transformation can take a substantial share of ML project work (Google for Developers, n.d).

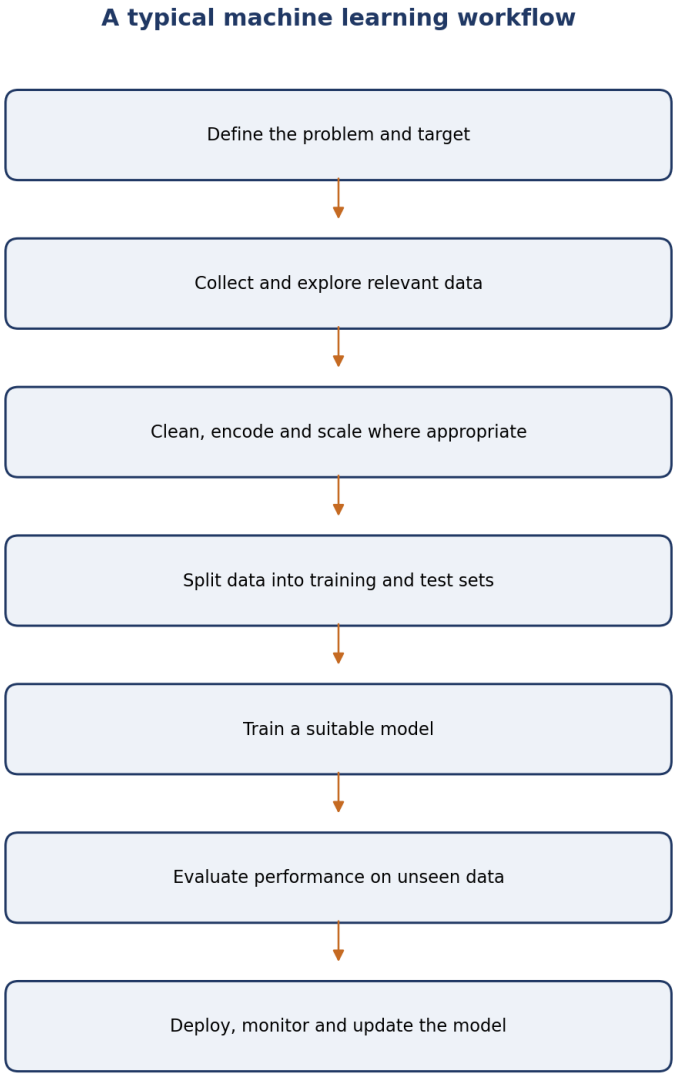


Figure 3. Typical machine learning workflow. Adapted from Google for Developers (n.d.-a) and scikit-learn developers (n.d.-a).

PART B – SUPERVISED VS UNSUPERVISED LEARNING

B1. What is supervised learning?

Supervised learning is when a model is trained using labelled examples. This means the training data contains both the input information (features) and the correct outcome (answers). The model learns the relationship between the features and the label(target) so it can predict the label for new records. For example, a credit risk dataset may contain borrower information (features) and a loan status column (label) indicating whether a borrower defaulted. The model learns from these known outcomes to predict potential loan defaults (Google for Developers, n.d.).

B2. What is unsupervised learning?

Unsupervised learning uses data without a supplied target label. Instead of being told the correct answer, the algorithm searches for patterns, similarities or groups in the data (Google for Developers, n.d.). For example, a financial institution could group customers according to their spending habits or transaction behaviour (patterns) without deciding the customer groups beforehand.

B3. Comparison of supervised and unsupervised learning

Comparison	Supervised learning	Unsupervised learning
Goal	Predict a known outcome or label.	Discover structure or patterns.
Data needed	Labelled data: features and known answers.	Unlabelled data: features without a target answer.
Typical tasks	Classification and regression.	Clustering and dimensionality reduction.
Algorithms	LinearRegression, LogisticRegression, DecisionTreeClassifier.	KMeans and PCA.
How success is measured	Compare predictions with known outcomes using suitable metrics.	Assess cluster quality, usefulness and whether patterns make sense in context.
Example	Predict whether a patient will be readmitted.	Group patients with similar characteristics without pre-defined groups.

The main difference is whether the correct answers are provided during training. The choice depends on the problem being investigated (Google for Developers, n.d.).

B4. Classification and regression

Classification is a supervised learning where the model predicts a category or class an observation belongs to. For example, a model may predict whether a patient will be readmitted (Yes/No), or whether an Email is spam or not spam. Regression is also supervised learning, but it predicts a numerical value. For example, a model may estimate a house's selling price or a learner's final percentage. Therefore, classification produces a category, while regression produces a number (Google for Developers, n.d.).

B5. Clustering and dimensionality reduction

Clustering is an unsupervised technique that groups records or data based on similarity. This technique simply looks for similar patterns among rows, and then sorts them into labeled groups. For example, a shop could use K-means algorithm to group customers with similar spending behaviour without having customer segments defined beforehand.

Dimensionality reduction on the other hand reduces the number of features (columns) while trying to retain useful information. This is particularly useful when a dataset is too complex to analyze, for instance, it has 50 columns or features, it reduces the number by either removing some columns or combining them, reduction makes it easier to look at the data and processes it faster. Principal Component Analysis (PCA) is one method of dimensionality reduction. It can help simplify a dataset with many related features, making it easier to explore or use in modelling (scikit-learn developers, n.d.).

B6. Classifying the five scenarios

Scenario	Type and task	Reason
Predict a house selling price from size and suburb.	Supervised – regression.	Past sale prices provide known numerical labels; the model predicts a price.
Group customers by spending habits, with no groups defined upfront.	Unsupervised – clustering.	There are no pre-existing group labels; the algorithm finds similar customers.
Decide whether an Email is spam.	Supervised – classification.	Examples can be labelled spam or not spam, and the model predicts a category.
Predict a learner's final mark from term marks.	Supervised – regression.	The target is a numerical mark, and historical examples provide known outcomes.
Find groups of products with similar sales patterns.	Unsupervised – clustering.	The aim is to discover groups based on similarities in sales, not predict a known

	label.

B7. Six scikit-learn classes

Class and module	Learning type	What it is used for
sklearn.linear_model.LinearRegression	Supervised – regression	Predicts a numerical target by fitting a linear relationship between features and the target.
sklearn.linear_model.LogisticRegression	Supervised – classification	Predicts class membership; despite its name, it is a classifier.
sklearn.tree.DecisionTreeClassifier	Supervised – classification	Uses decision rules arranged like a tree to assign a class.
sklearn.neighbors.KNeighborsClassifier	Supervised – classification	Classifies a record using the majority class among its nearest training examples.
sklearn.cluster.KMeans	Unsupervised – clustering	Groups observations into a chosen number of clusters.
sklearn.decomposition.PCA	Unsupervised – dimensionality reduction	Transforms features into a smaller set of principal components.

The module paths and descriptions are taken from the scikit-learn API reference (scikit-learn developers, n.d.).

PART C – PREPARING THE DATA

Data preparation means checking and transforming the dataset so that it is suitable for analysis and model training. I understand this as making sure the model receives information in a consistent and usable format. Pandas is useful for exploring and cleaning tables, while scikit-learn provides transformers and tools for preparing features (pandas development team, n.d.; scikit-learn developers, n.d.).

C1. What should be checked first?

When opening a new dataset, I first inspect its shape, column names, data types, summary statistics, missing values and duplicate records. This helps me understand the structure and quality of the data before making changes. For example, a numerical column stored as text may need to be converted into a numeric data type before it can be used for calculations.

In my credit risk dataset, I used inspection methods to understand the columns, identify numerical and categorical variables, examine summary statistics and check for unusual values. Useful pandas functions include df.shape, df.head(), df.info(), df.describe(), df.isnull().sum(), df.duplicated().sum() and df.astype() (pandas development team, n.d.).

C2. Handling missing values

Missing values occur when information is not recorded or is unavailable. They can be handled by removing incomplete rows or columns, or by filling the missing values through a process called imputation.

The mean can be used to fill missing numerical values when the data is reasonably balanced and not strongly affected by extreme values. The median is often more suitable for skewed numerical data because it is less affected by outliers. For categorical columns, the most frequent category (mode) or a constant value such as 'Unknown' may be appropriate.

Removing every row containing a missing value can result in losing a large amount of useful information, especially in smaller datasets. In my notebook, I used scikit-learn's SimpleImputer within preprocessing pipelines. The numerical features were prepared using median imputation, while categorical features used the most frequent value. This allowed the missing values to be handled consistently during model preparation. Pandas also provides dropna() and fillna() for handling missing values (pandas development team, n.d.; scikit-learn developers, n.d.).

C3. Encoding categorical data

Machine learning algorithms generally require numerical inputs rather than text categories. Encoding converts categorical information into a numerical format/data that an algorithm can process.

There are three common encoding methods:

- **Label encoding:** Assigns an integer to each category. It is commonly used for target labels. However, using arbitrary numbers for nominal input categories can incorrectly suggest that one category is greater than another.
- **Ordinal encoding:** Assigns numbers according to a meaningful order. For example, low = 0, medium = 1 and high = 2.
- **One-hot encoding:** Creates a separate binary column for each category. It is suitable for nominal categories such as housing type or loan purpose because these categories do not have a natural ranking.

Nominal categories have no meaningful order, while ordinal categories have an identifiable ranking. Choosing the wrong encoder can introduce misleading relationships into the data.

One-hot encoding can also create a very large number of additional columns when a feature contains hundreds of unique categories.

In my credit risk project, I used OneHotEncoder for categorical input features. This converted the text-based categories into numerical columns. The preprocessing process expanded my 11 original input features to 26 processed features. I did not encode loan_status as an input feature because it was the target variable, not a predictor (scikit-learn developers, n.d.).

C4. Scaling numerical features

Scaling adjusts numerical features to comparable ranges. This is important because some algorithms are affected by differences in feature sizes. For example, income may be recorded in thousands while age is recorded in tens. Without scaling, some algorithms may give greater influence or weight to features with larger numerical values and less to smaller ones.

Two common methods are:

- **Normalisation (Min-Max scaling):** Transforms values to a specified range, commonly between 0 and 1. . The absolute lowest value becomes 0, and the highest becomes 1, and everything else falls in between.
- **Standardisation (z-score scaling):** Transforms values so that they have a mean of approximately 0 and a standard deviation of approximately 1.

Algorithms such as K-Nearest Neighbours, Support Vector Machines and Logistic Regression generally benefit from scaling. Decision Trees and Random Forests generally do not require it because their decisions are based on feature thresholds rather than distances or feature magnitudes.

In my notebook, I used StandardScaler on the numerical input features. This was included in the preprocessing pipeline so that the scaling parameters were learned from the training data and then applied consistently to the test data (scikit-learn developers, n.d.).

C5. Splitting into training and test sets

A train-test split divides a dataset into two parts. The training set is used to teach the model patterns in the data, while the test set is kept separate to evaluate how well the trained model predicts outcomes it has not seen before.

An 80/20 split is commonly used, meaning 80% of the records are used for training and 20% for testing. The scikit-learn function train_test_split() performs this task. The random_state

parameter makes the split reproducible, while stratify=y helps maintain approximately the same proportions of target classes in both sets.

In my project, I used an 80/20 stratified split. The dataset contained 32,416 records, resulting in 25,217 training records and 6,305 testing records. Stratification was important because the target variable was imbalanced: approximately 78.41% of records represented no default (0), while 21.59% represented default (1).

Data leakage happens when information from the test set influences the training process, resulting in an unreliable evaluation. To prevent this, the data must be split before fitting preprocessing transformations. The fit() method learns the required settings, transform() applies the learned settings, and fit_transform() learns and applies the transformation in one step. In my notebook, the preprocessing pipeline was fitted on the training data and then applied to the test data. This helped prevent data leakage (scikit-learn developers, n.d.).

C6. Identifying and dealing with outliers

Outliers are values that are very different from most of the other values in a dataset, they are unusually far from most observations. For example, if most customers spend between R100 and R1,000 but, one customer spends R50,000, that value is an outlier.They can be identified using boxplots, the interquartile range (IQR) method or z-scores.

An outlier is not automatically an error. For example, recording a person's age as 190 could indicate a data-entry mistake. However, an unusually high income or loan amount may be genuine and important information. Removing valid outliers without investigating them can cause useful information to be lost.

In my credit risk dataset, I investigated unusually high values in numerical columns. The checks identified five records with a person age above 100 and two records with employment length above 50 years. These values required investigation because they could represent data-entry errors or unusual but genuine observations. The decision to remove, correct or retain an outlier should be based on the context of the data rather than automatically deleting every unusual value.

Pandas provides quantile() for calculating quartiles and clip() for limiting values to specified boundaries. Visualisation tools such as Seaborn's boxplot() can also help identify potential outliers (pandas development team, n.d.; scikit-learn developers, n.d.).

C7. Feature engineering

Feature engineering is when new columns or input features are created from already existing features in a dataset. This makes the data more useful and clearer when training a machine learning model. For example, a transaction date can be used to create a month or day-of-week feature. A person's date of birth can also be used to calculate their age.

A correlation matrix helps examine relationships between numerical features. A strong positive or negative correlation may indicate that two features are closely related. This can help identify potentially redundant variables, although correlation alone does not prove that a feature should be removed.

In a credit risk project, feature engineering could involve creating a ratio between a borrower's loan amount and income. Such a feature may provide additional information about the size of the loan relative to the borrower's earnings. Pandas provides pd.to_datetime(), the .dt accessor and df.corr() to support date-based feature creation and correlation analysis (pandas development team, n.d.).

Feature engineering is important because it helps organize data in a way that makes it easier for a model to identify useful, relevant patterns to make predictions.

Practical application using pandas and scikit-learn

I applied the data preparation concepts in a Databricks notebook using my credit risk dataset. I loaded and explored the data, checked its structure and unusual values, separated the predictors (X) from the target (y), and performed a stratified 80/20 train-test split.

I then created a preprocessing workflow using ColumnTransformer and Pipeline. Numerical features were imputed using the median and scaled using StandardScaler. Categorical features were imputed using the most frequent category and converted using OneHotEncoder. The preprocessing workflow was fitted on the training data and applied to the test data using the same learned transformations.

After preprocessing, the training and testing datasets each contained 26 transformed input features. I then trained and evaluated classification models using the prepared data. This practical work demonstrated how raw data can be transformed into a format suitable for machine learning while reducing the risk of data leakage.

PART D – FURTHER RESEARCH

The following five topics were selected because they connect directly to model building and the preparation of patient readmission data.

D1. Overfitting and underfitting

Overfitting and underfitting are problems that can happen when training a machine learning model.

Overfitting happens when a model learns the training examples too closely, including redundant details instead of learning general patterns. This means that the model may perform well on the historical data it was trained with but, struggle when new information is given. Google explains that overfitting can be caused by a model that is too complex or by training data that does not represent the real situation well (Google for Developers, n.d.).

Underfitting happens when a model is too simple to learn and understand the important relationships in the data (patterns). It may therefore produce poor results during both training and test data.

For example, in a loan-default prediction project, an overfitted model may memorise patterns in existing loan applications but fail to identify default risk correctly for new applicants. An underfitted model may fail to recognise useful relationships between the available financial features and loan defaults.

If training performance is very high but test performance is much lower, overfitting may be occurring. If both are poor, underfitting may be a possibility. Possible responses include collecting more representative data, reducing model complexity, selecting better features or using regularisation. The aim is not to memorise the training data but to generalise to new cases.

D2. Measuring a model

After training a machine learning model, it is important to check whether it is giving good results or not. This is called measuring model performance. It is applied on both types of Supervised Learning, that is, classification and regression.

For classification, some common measurements are accuracy, precision, recall, and the F1-score:

- Accuracy is the proportion of all predictions that are correct; essentially, it shows how many predictions are correct,
- Precision measures how many cases predicted as positive are actually positive,
- Recall measures how many actual positive cases the model manages to successfully identify,
- The F1-score combines precision and recall into one

A confusion matrix is also useful because it shows correct (true positives, true negatives) and incorrect (false positives and false negatives) made by a model (Google for Developers, n.d.).

Accuracy alone can be misleading when the target classes are imbalanced. In my loan-default dataset, approximately 78.41% of records represented no default (0), while 21.59% represented default (1). A model could achieve high accuracy by predicting the majority class more often while still missing many actual defaults.

I compared Logistic Regression, Decision Tree and Random Forest using the test set. Random Forest achieved the highest accuracy of 93.31% and F1-score of 82.21%. Its precision was 96.42%, while recall was 71.66%. This means it correctly identified many of its predicted default cases, but it still missed some actual defaults. Therefore, precision, recall, F1-score and the confusion matrix are important alongside accuracy when evaluating a loan-default classification model.

For regression, there are different measures. Mean Absolute Error (MAE) measures the average difference between predicted and actual values. Root Mean Squared Error (RMSE) gives more weight to larger errors, and R-squared R^2 shows how much of the variation in the target(results) is explained by the model.

D7. Class imbalance

Class imbalance occurs when one target category has significantly more records than another. This can mislead a machine learning model because it may learn to predict the majority class more frequently and still achieve a high accuracy score, while failing to identify enough minority-class cases.

In my credit risk dataset, the target variable loan_status contains two classes: 0, meaning no loan default, and 1, meaning loan default. The dataset contains 24,715 records with no default (78.41%) and 6,807 records with default (21.59%). This shows that the classes are imbalanced, with no default being the majority class.

There are several ways to address class imbalance. Oversampling increases the representation of the minority class, while undersampling reduces the number of majority-class records. Class weighting gives greater importance to errors involving the minority class. Another method is SMOTE, which creates synthetic minority-class examples based on existing records. However, resampling and SMOTE must only be applied to the training data to avoid data leakage.

In my project, I used stratified train-test splitting to maintain approximately the same class proportions in both datasets. I also compared precision, recall and F1-score instead of relying only on accuracy. These measures help assess how well the models identify loan defaults (Google for Developers, n.d.; scikit-learn developers, n.d.).

D9. Pipelines

A machine learning pipeline is a way of connecting data preparation steps so that the data can be prepared and used to train a model in an organized way. This is useful because real datasets often require several transformations before they can be used by a model. For example, numerical features may need missing values filled and values scaled, while categorical features may need missing values filled and categories encoded.

Scikit-learn provides Pipeline, which connects steps in a specific order, and ColumnTransformer, which allows different preprocessing methods to be applied to different columns. In a credit risk project, numerical loan-related features can be processed using median imputation and standardisation, while categorical features can be processed using most-frequent imputation and one-hot encoding.

I used both tools in my notebook. The ColumnTransformer handled numerical and categorical features separately, while the pipeline ensured that preprocessing was applied consistently before model training. The preprocessing steps were fitted using training data and then applied to the test data. This helps prevent data leakage and avoids forgetting to apply the same transformations to unseen records.

Pipelines are also useful during cross-validation because preprocessing can be repeated within each training fold. This makes the modelling process more organised, consistent and reproducible. However, a pipeline does not automatically make a model accurate; the selected preprocessing steps and model still need to be evaluated (scikit-learn developers, n.d.).

D11. Data protection and POPIA

POPIA stands for the Protection of Personal Information Act 4 of 2013. It is a South African law that regulates how organisations collect, use, store and protect personal information. It is important in machine learning because datasets used to train models may contain personal and financial information, such as names, identity details, income and loan records (Republic of South Africa, 2013).

In credit risk modelling, organisations should have a lawful reason for processing personal information and should only collect information that is necessary for the intended purpose. They should also protect the information from unauthorised access and use appropriate security measures.

Personal information is information that identifies a person or can reasonably be linked to them. Anonymised information has been changed so that an individual cannot reasonably be identified. However, removing names alone may not be enough because combinations of information could still identify someone. Pseudonymised information replaces direct identifiers with codes, but it may still be possible to identify the individual if the code can be linked back to them.

When developing a loan-default prediction model, financial institutions should consider privacy, fairness and transparency. A prediction should not automatically determine whether a person receives credit. Human oversight is important because incorrect or biased predictions can negatively affect applicants. Organisations must therefore handle personal information responsibly and comply with POPIA (Republic of South Africa, 2013).

Conclusion

This research explored the application of Machine Learning in predicting loan defaults using a credit risk dataset. The purpose was to understand how machine learning can use historical borrower information to identify patterns and predict whether a borrower is likely to default on a loan. The research also examined the different types of machine learning, data preparation techniques, classification algorithms and model evaluation methods.

The practical implementation was conducted using Python, Pandas and Scikit-learn in Databricks. The dataset contained 32,416 records after preparation, with 11 predictor variables and a target variable called loan_status. The target represented two outcomes: 0 for borrowers who did not default and 1 for borrowers who defaulted. Data preparation included investigating missing values, examining outliers, separating the predictors from the target, splitting the data into training and testing sets, and applying imputation, encoding and scaling through a preprocessing pipeline.

Three supervised classification algorithms were developed and evaluated: Logistic Regression, Decision Tree and Random Forest. Their performance was assessed using accuracy, precision, recall, F1-score, confusion matrices and five-fold stratified cross-validation. Random Forest achieved the strongest overall performance, with a test accuracy of 93.70% and an F1-score of 83.28%. Its cross-validation F1-score was 82.21%, suggesting relatively consistent performance across the validation folds. However, Decision Tree achieved the highest recall, indicating that it identified a larger proportion of actual loan defaults.

The findings demonstrate that machine learning can support credit risk analysis by identifying patterns associated with loan defaults. However, model accuracy alone is not sufficient to establish whether a model is suitable for real lending decisions. The consequences of false predictions, fairness, data quality and responsible use must also be considered.

Overall, this research provided practical experience in preparing data, developing classification models and evaluating their performance. It also demonstrated how machine learning can contribute to financial decision-making when implemented responsibly and supported by appropriate human oversight.

REFERENCES

1. Google for Developers (n.d.) Machine Learning Crash Course. Available at: https://developers.google.com/machine-learning/crash-course (Accessed: 25 October 2026).
2. IBM (n.d.) Machine learning. IBM Think. Available at: https://www.ibm.com/think/machine-learning (Accessed: 1 November 2026).
3. Nkosi, S. (2025) An explainable machine-learning approach to credit scoring. Master's dissertation, University of Johannesburg. Available at: https://hdl.handle.net/10210/517449 (Accessed: 1 November 2026).
4. Omara, T. et al. (2023) A field-based recommender system for crop disease detection using machine learning. Available at: https://pubmed.ncbi.nlm.nih.gov/37181731/ (Accessed: 27 October 2026).
5. pandas development team (n.d.) pandas documentation. Available at: https://pandas.pydata.org/docs/ (Accessed: 1 November 2026).
6. Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer. Available at: https://www.gov.za/documents/protection-personal-information-act (Accessed: 30 October 2026).
7. scikit-learn developers (n.d.) Scikit-Learn Documentation. Available at: https://scikit-learn.org (Accessed: 1 November 2026).
8. Gomez Selvaraj, M., Vergara, A., Montenegro, F., Alonso Ruiz, H., Safari, N., Raymaekers, D., Ocimati, W., Ntamwira, J., Tits, L., Omondi, A.B. and Blomme, G. (2020) Detection of banana plants and their major diseases through aerial images and machine learning methods: A case study in DR Congo and Republic of Benin. ISPRS Journal of Photogrammetry and Remote Sensing, 169, pp. 110–124. Available at: https://doi.org/10.1016/j.isprsjprs.2020.08.025 (Accessed: 1 November 2026).

AI USE DECLARATION

Artificial Intelligence (AI) tools, including ChatGPT and Google search engine, were used as supporting resources during this research assignment. The tools assisted with understanding machine learning concepts, explaining Python code, developing and troubleshooting sections of the practical Databricks notebook, and interpreting model evaluation results.

AI assistance was also used to support the organisation and drafting of certain sections of the research report. I remained responsible for selecting the research topic and dataset, conducting the practical implementation, executing the code, reviewing the outputs and interpreting the findings.

The AI-generated suggestions and explanations were reviewed and adapted to reflect the research objectives, assessment requirements and actual results obtained from the practical implementation. I reviewed the explanations against the cited sources and am responsible for checking, editing and submitting the final work in my own words. I also used official pandas and scikit-learn documentation to verify the technical tools and concepts.

DECLARATION OF AI ASSISTANCE

I used ChatGPT and Google search engine to help me understand the assignment questions, Python functions covered in the assignment, organise research notes, simplify technical explanations and prepare an initial draft structure. I reviewed the explanations against the cited sources and am responsible for checking, editing and submitting the final work in my own words. I also used official pandas and scikit-learn documentation to verify the technical tools and concepts.